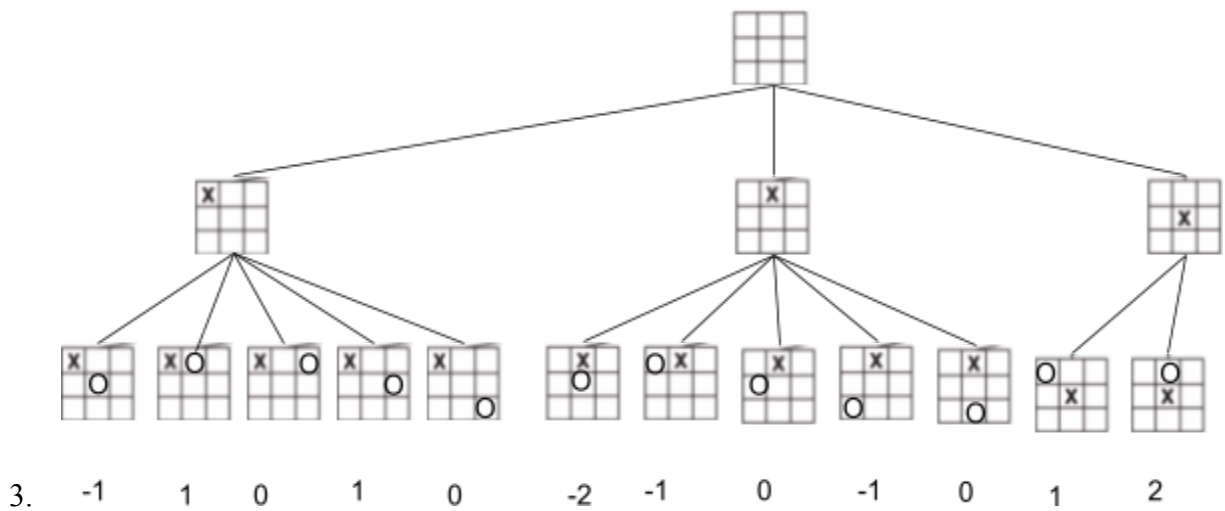
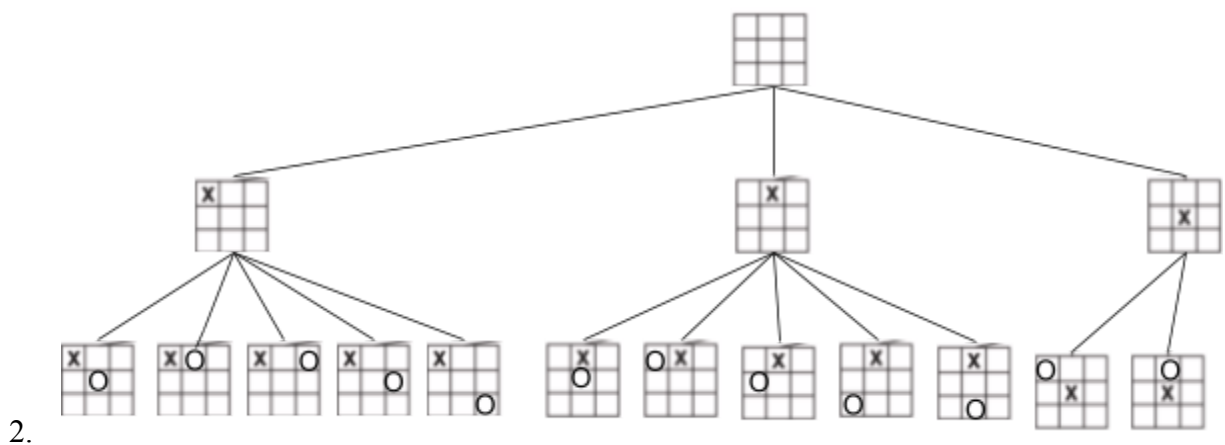


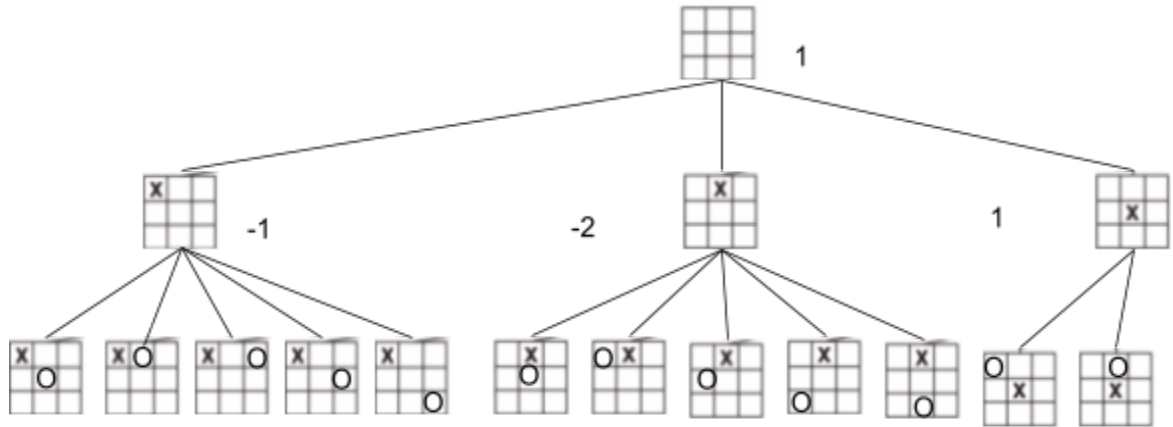
Elvis Cao
 Professor Grimes
 CS 380
 26 Feb 2025

Assignment 4 Written Problems

1. The tic-tac-toe board has 9 tiles in a 3x3 board and after each move there are 9 moves initially, then 8, 7, 6, and so on meaning there are approximately 9! games. Calculating this value, 9! is equal to 362,880 games.

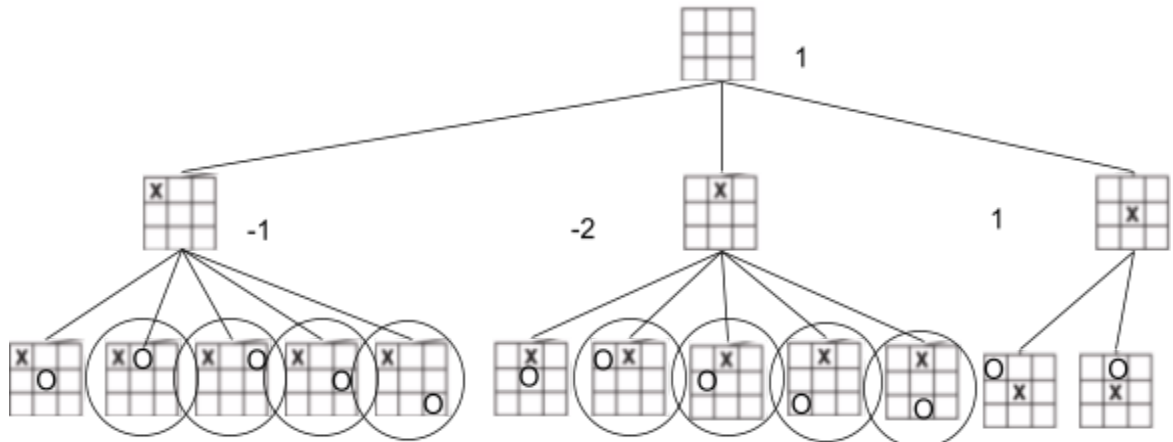


Using the evaluation function $Eval(s) = 3X2(s) + X1(s) - (3O2(s) + O1(s))$, I substituted in the corresponding values for each distinct state at a depth of 2 and got the values as shown below each state in the diagram.



4. -1 1 0 1 0 -2 -1 0 -1 0 1 2

At a depth of 1, O wants to minimize the values from each of the sets of states in depth 2, so I got -1, -2, and 1 respectively. At a depth of 0, as X is making the first move, X will take the highest value from depth 1, which means it will have a value of 1 at the root.



5. -1 1 0 1 0 -2 -1 0 -1 0 1 2

We would first need to evaluate the right side, as it is the best path to go for the maximizer X, so both nodes in depth 2 on the right will be evaluated which will update 1 to both the right node at depth 1 and the root node as 1. Then we go down the left side and see that -1 is already the minimum so we do not need to visit the remaining 4 nodes in depth 2 on the left side as these all have values greater than the min of -1. Lastly, we go down the middle and visit the -2 state and this is the minimum so we do not need to visit the remaining 4 nodes for the middle at depth 2 as they are all greater than the min of -2. I circled the 8 nodes that can be pruned in the diagram above.

Assignment 4 Documentation

To begin with, I first worked on implementing a random agent that plays othello by selecting moves at random. This was modified in the agent.py in the RandomAgent class where I used random.chooseMoves to select each move randomly. The next agent I implemented was the minimax agent which uses the minimax algorithm to select moves that would best maximize the gains in Othello. This was modified in agent.py too under the MinimaxAgent class. The MinimaxAgent explores all possible game states up to a certain depth, specified in the command line as `sh run.sh minimax random <depth>` which is a number of the depth to which it searches for each move it makes. Since it uses the minimax standard algorithm, the time complexity is approximately $O(b^d)$, where b is the branching factor and d is the depth of how deep it searches. I also implemented the alpha beta agent modified under the AlphaBeta class in agent.py. This agent builds off minimax by incorporating alpha-beta pruning, which reduces the search space by eliminating branches that will not affect the final decision. AlphaBeta outperforms Minimax due to the added pruning technique, the worst time complexity is still the same as Minimax which is $O(b^d)$, but the average time complexity is closer to $O(b^{(d/2)})$, which is a better runtime than Minimax, in general. Alphabeta can search faster than Minimax when given the same depth to search. I also implemented the extra credit agent which is written in my ehc35.py file, which searches using the time that it is given in milliseconds (at least 100 milliseconds is the constraint given in the instructions), and in this time it can start searching at depth 1, then 2, 3, so on as long as it is within the time allotted. It harnesses the technique of iterative deepening alpha beta search. Basically, it starts at a shallow depth and keeps searching within the time frame, which ensures it selects the best move. The evaluate function enhances the basic score function that is given by prioritizing edge and corner positions. This agent aims to maximize performance by combining IDS and alpha-beta pruning. Below are some testing of my agents: Minimax defeats random at a depth of 3, on average it wins at least 90% of games based on repeated iterations, as the O player:

```
ehc35@tux2:~/CS380/A4$ sh run.sh minimax random 3|
```

```
*** Final winner: O ***
```

```
X O X X X X X X
O O O X X X X X
O O X O O O X X
O O O O O O O X
O O O X X O O X
O O O O O X O X
O O O O X O O X
O O O O O O O X
```

```
ehc35@tux2:~/CS380/A4$ |
```

Minimax can also play as the second player who wins as the X player, as denoted in the instructions:

```
ehc35@tux2:~/CS380/A4$ sh run.sh random minimax| 3
```

```
*** Final winner: X ***
```

```
X X X X X X X O
X X X X X X X X
X X X X X O X X
X X X X O X X X
X O O O X X X X
X X X X O X X X
X X X X X X X X
O X X X X X X X
```

Alphabeta also defeats random over 90% of games, with repeated iterations as an O player:

```
ehc35@tux2:~/CS380/A4$ sh run.sh alphabeta random 3|
```

```
*** Final winner: O ***
```

```
O . . O . . . .  
. O . O O O . .  
. O O O O O . .  
. . . O O O O O  
O O O O O O O .  
. O O O O O O .  
O O . O . . O .  
O . . O . . . .
```

Alpha Beta can also play as the second player, as X, and defeat random too:

```
ehc35@tux2:~/CS380/A4$ sh run.sh random alphabeta 3
```

```
*** Final winner: X ***
```

```
. . . . X . . .  
. . X X X . . .  
. X X X X . . .  
X X X X X X X .  
. X . X X . . .  
. . X X X . . .  
. . X X . . . .  
. X . . . . . .
```

Lastly, my extra agent named ehc35 can also defeat random with a high probability of 90%+, when given the bare minimum of 100ms, ehc35 is the O player as it is the first player here (lowest constraint, higher times mean better moves):

```
ehc35@tux2:~/CS380/A4$ sh run.sh extra random 100
```

```
*** Final winner: 0 ***
0 0 0 0 0 0 0 X
0 0 0 X X X X 0
0 0 0 0 X X X 0
0 0 0 0 0 X X 0
0 0 0 X 0 0 X 0
0 0 0 X 0 X 0 0
0 0 X 0 0 0 0 0
X X 0 0 0 0 0 0
```

My extra agent ehc35 can also play as the second player, X, and defeat random:

```
ehc35@tux2:~/CS380/A4$ sh run.sh random extra 100|
```

```
*** Final winner: X ***
0 0 0 X X X X X
0 0 0 0 X X X X
0 X 0 X 0 X 0 X
0 X 0 0 X 0 X X
X X 0 0 0 X 0 X
X X X 0 0 0 X X
X X X X X X 0 X
X X X X X X X 0
```